

SENAI Dr. Celso Charuri - Uberlândia/MG

Instrutor: Lucas Lopes

Disciplina: Desenvolvimento Full-stack / Testes de Backend

Atividade Full-Stack: Landing Page e Testes de Integração (Projeto Saga)

Nesta atividade, vamos conectar as duas pontas do desenvolvimento de software. Vocês irão atuar no projeto "Saga" tanto na construção visual da aplicação (Front-end) quanto na garantia de qualidade da API (Back-end).

O código base que utilizaremos como referência já está disponível. Vocês devem acessar o repositório da turma no link abaixo:

github.com/lucaslopes-ti/senaihub

<https://github.com/lucaslopes-ti/senaihub>

PARTE 1: Front-end (React + SCSS)

Sua primeira missão é desenvolver a página inicial (Landing Page) do projeto utilizando React e estilizá-la com SCSS para garantir um visual moderno e institucional.

O que observar no código de referência:

- **A Estrutura do Home.jsx:** Note o uso de semântica HTML (<header>, <section>, <footer>) e como integramos componentes do Bootstrap (ex: Carousel e Grid System) dentro do React.
 - **Estilização no custom.scss:**
 - **Glassmorphism (Efeito Vidro):** Combinação de fundo translúcido rgba() com backdrop-filter: blur(8px).
 - **Gradientes:** Fundos vibrantes criados misturando linear-gradient e radial-gradient.
 - **Micro-animações:** Uso de @keyframes (como o techFadeUp) para que os elementos apareçam de forma suave ao carregar a página.
-

PARTE 2: Back-end (Testes de Integração)

Depois de construir a interface, precisamos garantir que o nosso Back-end está respondendo corretamente. Na segunda etapa, vocês farão **Testes de Integração** na camada de *Controller* do Spring Boot.

Os testes de integração simulam uma requisição HTTP real (a mesma que o seu Front-end fará) e verificam se as rotas, serviços e banco de dados estão integrados.

Código Guia para o Teste de Integração (MockMvc)

Aqui está um exemplo de como testar a criação de uma Notícia simulando uma requisição POST:

```
package com.ipi.senaihub;

import com.ipi.senaihub.model.Noticia;
import com.ipi.senaihub.service.NoticiaService;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.boot.test.autoconfigure.web.servlet.AutoConfi
gureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static
org.springframework.test.web.servlet.request.MockMvcRequestBuil
ders.post;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatche
rs.status;

@SpringBootTest
@AutoConfigureMockMvc // Habilita o MockMvc para simular
requisições HTTP
public class NoticiaControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean // "Finge" o serviço para isolar o teste no Controller
    private NoticiaService noticiaService;
```

```
@Test
public void deveRetornarStatusCriadoAoPublicarNoticia()
throws Exception {
    // 1. Simula o Payload JSON enviado pelo Front-end
    String jsonRequest = "{ \"titulo\": \"Nova Notícia\", \"conteudo\": \"Conteúdo\" }";

    // 2. Simula a resposta do Serviço

    Mockito.when(noticiaService.publicarNoticia(Mockito.any(Noticia.class)))
        .thenReturn(new Noticia());

    // 3. Faz a requisição POST simulada e verifica se retorna 201 Created
    mockMvc.perform(post("/api/noticias")
        .contentType(MediaType.APPLICATION_JSON)
        .content(jsonRequest)
        .andExpect(status().isCreated()));
}
```

3. O Desafio (O que entregar)

Acessem o repositório do Github, estudem os códigos e realizem as entregas abaixo no projeto de vocês:

Entrega Front-end:

1. **Home.jsx**: Crie a Landing Page contendo:
 - Um Navbar superior (com logo e links).

- Uma Seção Hero de grande impacto (com gradientes de fundo).
 - Uma Seção de Destaques (usando Grid de 3 colunas para os cards).
 - Um Rodapé (Footer).
2. **custom.scss:** Aplique a estilização moderna. É obrigatório o uso do efeito *Glassmorphism* em algum painel ou card e animações de entrada (@keyframes) nos blocos de texto.

Entrega Back-end (Testes):

1. Escolha as entidades do projeto (ex: Curso ou Usuario).
2. Crie uma classe de **Teste de Integração** (ex: CursoControllerIntegrationTest.java) utilizando o MockMvc.
3. Escreva pelo menos um teste simulando o envio de dados via POST (criando um registro) ou GET (buscando um registro) garantindo que a resposta HTTP (200 OK ou 201 Created) retorne **Verde** na execução.

Dica do Instrutor: Pegue um cafezinho e caprichem nas duas pontas! Bom código!